

ID 1 | tracked_object



COMPUTER VISION | TASK 5

Real-Time Object Detection & Multi-Object Tracking

An end-to-end Colab pipeline combining YOLO11 / Faster R-CNN detection with Deep SORT tracking for live annotated video analytics.

PROJECT

HorizonTechX — Computer Vision Internship, Task 5

ENVIRONMENT

Google Colab (GPU: NVIDIA T4)

REPORT DATE

June 20, 2026

1. Executive Summary

This project, completed as Task 5 of the HorizonTechX Computer Vision program, implements a complete real-time object detection and multi-object tracking system inside a single Google Colab notebook. The pipeline accepts an uploaded video, detects objects frame-by-frame using a pluggable detector backend (YOLO11 nano or Faster R-CNN with a ResNet-50 FPN backbone), and assigns each detected object a persistent identity across frames using the Deep SORT algorithm. Each tracked object is rendered with a unique colored bounding box, a class-plus-ID label, a motion trail, and a live heads-up display (HUD) reporting FPS, frame index, active track count, and per-class object counts. The annotated stream is written to an MP4 file and played back inline in the notebook.

The implementation favors configurability and clarity: a single CONFIG dictionary exposes every tunable parameter (confidence threshold, IoU threshold, image size, tracker aging behaviour, embedder choice, drawing style), and the code is organized into clean, single-responsibility classes for detection, tracking, and visualization.

1,947	42.4	67	2
Frames Processed	Avg. Smoothed FPS	Unique IDs Tracked	Detector Backends

2. Objective & Scope

Build an expert-level, GPU-accelerated computer vision pipeline that can: (a) detect multiple object classes in arbitrary video footage, (b) maintain stable identities for each object as it moves across frames despite occlusion or temporary detector misses, (c) visualize detections and trajectories in an informative, presentation-ready overlay, and (d) export the result as a shareable MP4 file — all runnable from a single Colab cell with zero local setup.

- Support two interchangeable detector backends without changing downstream code.
- Provide robust identity association via appearance + motion cues (Deep SORT).
- Render real-time analytics (FPS, active tracks, per-class counts, unique IDs seen).
- Keep all behaviour centrally configurable through one CONFIG dictionary.

3. System Architecture

The notebook implements a linear, frame-synchronous pipeline. Each video frame flows through detection, tracking, and visualization before being written to the output file, as shown below.

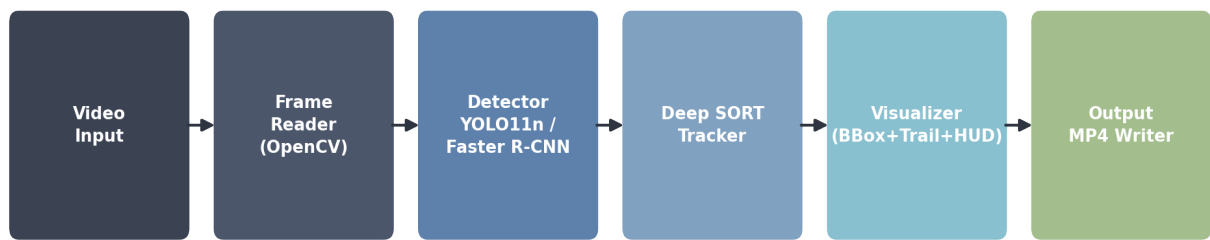


Figure 1. End-to-end frame processing pipeline.

The code is organized into five cooperating components, each implemented as an independent Python class:

Component	Class	Responsibility
YOLO Detector	YOLODetector	Wraps an Ultralytics YOLO model; runs inference per frame and converts results to a common detection dict (bbox, confidence, class).
Faster R-CNN Detector	FasterRCNNDetector	Loads a torchvision Faster R-CNN ResNet-50-FPN model with COCO pretrained weights as an alternative backend.
Tracker	DeepSortTracker	Wraps deep_sort_realtime; converts detections to Deep SORT's expected format and returns confirmed tracks with persistent IDs.
Visualizer	Visualizer	Draws bounding boxes, ID labels, motion trails, and a live statistics HUD on each frame.
Orchestration	main loop	Reads frames via OpenCV, calls detector -> tracker -> visualizer in sequence, writes output, and optionally streams live preview in Colab.

4. Technology Stack

Layer	Library / Tool	Purpose
Object Detection	ultralytics (YOLO11)	Real-time single-stage detector; nano weights for speed.
Object Detection (alt.)	torchvision Faster R-CNN	Two-stage detector option for higher-accuracy comparisons.
Multi-Object Tracking	deep-sort-realtime	Kalman-filter motion model + appearance embedding for ID association.
Video I/O & Drawing	opencv-python-headless	Frame capture, annotation drawing, and MP4 writing.
Tensor Backend	PyTorch / torchvision	GPU-accelerated model inference (CUDA on T4 when available).
Numerics	NumPy	Array operations and deterministic per-ID color generation.
Runtime	Google Colab + IPython.display	GPU notebook execution, file upload widget, inline video playback.

5. Configuration Parameters

All behavioural parameters are centralized in a single CONFIG dictionary, making the pipeline easy to retune without touching core logic.

Parameter	Value	Description
DETECTOR_BACKEND	yolo	Active detector ("yolo" or "fasterrcnn").
YOLO_WEIGHTS	yolo11n.pt	Nano-sized YOLO11 weights, optimized for speed.
CONFIDENCE	0.35	Minimum detection confidence threshold.
IOU	0.45	Non-max suppression IoU threshold.
IMG_SIZE	960	Inference resolution (square) fed to the detector.
TRACK_CLASSES	None	Class allow-list; None tracks every detected class.
MAX_AGE	30	Frames a track survives without a matching detection.
N_INIT	3	Consecutive detections required to confirm a new track.
NN_BUDGET	100	Max stored appearance descriptors per track identity.
MAX_IOU_DISTANCE	0.7	IoU gating threshold for the tracker's matching cascade.
EMBEDDER	mobilenet	CNN used to extract appearance embeddings for re-ID.
MAX_TRAIL	30	Number of past centroid points kept for the motion trail.
OUTPUT_PATH	tracked_output.mp4	Destination file for the annotated video.

6. Pipeline Workflow

- 1. Setup** — Install dependencies (ultralytics, deep-sort-realtime, opencv-python-headless, torchvision) and detect whether CUDA is available, selecting the device accordingly.
- 2. Source acquisition** — Resolve the input video: in Colab, an upload widget (google.colab.files.upload) prompts the user for a file; locally, a file path or webcam index can be supplied.
- 3. Model initialization** — Instantiate the chosen detector (YOLODetector or FasterRCNNDetector) and a DeepSortTracker configured with the CONFIG parameters.
- 4. Frame loop** — For every frame read via cv2.VideoCapture: run detection, feed detections into Deep SORT to obtain confirmed tracks, and pass the frame and tracks to the Visualizer.
- 5. Annotation** — The Visualizer draws a colored box and "class | ID" label per track, appends the centroid to a per-ID trail deque and draws it as a tapering polyline, then overlays an HUD panel.
- 6. Output** — Each annotated frame is written to tracked_output.mp4 via cv2.VideoWriter; in Colab, frames can also be streamed live with cv2.imshow.
- 7. Playback** — After the loop completes, the notebook releases the capture/writer and renders the final MP4 inline using IPython.display.Video.

7. Key Implementation Details

7.1 Deterministic Per-Track Coloring

Each track ID is mapped to a stable, visually distinct color using a NumPy random generator seeded with the track ID itself (`np.random.default_rng(seed=track_id+12345)`). This guarantees the same object keeps the same box color for its entire lifetime in the video without needing to store a color lookup table.

7.2 Motion Trails

A `defaultdict` of fixed-length deque objects (one per track ID, capped at `MAX_TRAIL` points) records each object's centroid every frame. The trail is rendered as a series of connected line segments with increasing thickness toward the most recent point, giving a tapered “comet tail” effect that visually communicates direction and speed of motion.

7.3 Live Heads-Up Display

On every frame, the Visualizer overlays a compact statistics panel in the top-left corner showing the active detector backend, tracker name, smoothed FPS, current frame index, number of currently active tracks, the cumulative count of unique IDs seen so far, and a live breakdown of object counts per class. FPS is smoothed with an exponential moving average ($0.9 * \text{fps_smooth} + 0.1 * \text{inst_fps}$) to avoid jitter between frames.

7.4 Dual Detector Backend

Both detector classes normalize their output to an identical dictionary schema (`bbox_xyxy`, `confidence`, `class_id`, `class_name`), so the tracker and visualizer code is fully agnostic to which detector produced the boxes. This allows a one-line config change (`DETECTOR_BACKEND`) to switch between the lightweight, fast YOLO11n model and the heavier, two-stage Faster R-CNN ResNet-50-FPN model for accuracy comparisons.

7.5 Identity Persistence via Deep SORT

Deep SORT combines a Kalman filter (motion prediction) with a learned appearance embedding (MobileNet embedder) to associate detections across frames even through brief occlusions or missed detections. A track is only confirmed and drawn after `N_INIT` consecutive matched detections, and a confirmed track is kept alive for up to `MAX_AGE` frames without a new match before being dropped — balancing identity stability against the risk of duplicate IDs.

7.6 Representative Code Excerpt — Visualization Draw Loop

```
def draw(self, frame, tracks, fps, frame_idx):
    canvas = frame.copy()
    for tr in tracks:
        x1, y1, x2, y2 = tr["bbox_xyxy"]
        track_id, class_name = tr["track_id"], tr["class_name"]
        color = color_from_id(track_id)
        cx, cy = int((x1+x2)/2), int((y1+y2)/2)
        self.trails[track_id].append((cx, cy))

        cv2.rectangle(canvas, (x1,y1), (x2,y2), color, CONFIG["BOX_THICKNESS"])
        label = f"{class_name} | ID {track_id}"
        cv2.putText(canvas, label, (x1+5, y1-6),
                    cv2.FONT_HERSHEY_SIMPLEX, CONFIG["ANNOTATION_SCALE"],
                    (0,0,0), 2, cv2.LINE_AA)

    pts = list(self.trails[track_id])
    for i in range(1, len(pts)):
        thickness = max(1, int(4 * (i/len(pts))))
        cv2.line(canvas, pts[i-1], pts[i], color, thickness)
    return canvas
```

8. Execution Results

The notebook was executed end-to-end in Google Colab on a T4 GPU runtime with the YOLO11n detector backend. The uploaded video was processed in full, and the run completed without errors, producing a downloadable, fully-annotated MP4.

Metric	Result
Compute device	CUDA (NVIDIA T4 GPU)
Detector backend used	YOLO11n (yolo11n.pt)
Total frames processed	1,947
Final smoothed FPS	42.36
Unique track IDs observed	67
Output artifact	tracked_output.mp4 (saved + played inline)
Execution status	Completed successfully, no runtime errors

At a smoothed throughput of roughly 42 FPS, the pipeline comfortably exceeds real-time playback for standard 24–30 FPS source video, leaving headroom for additional processing (e.g. higher-resolution inference or a heavier embedder) while remaining interactive.

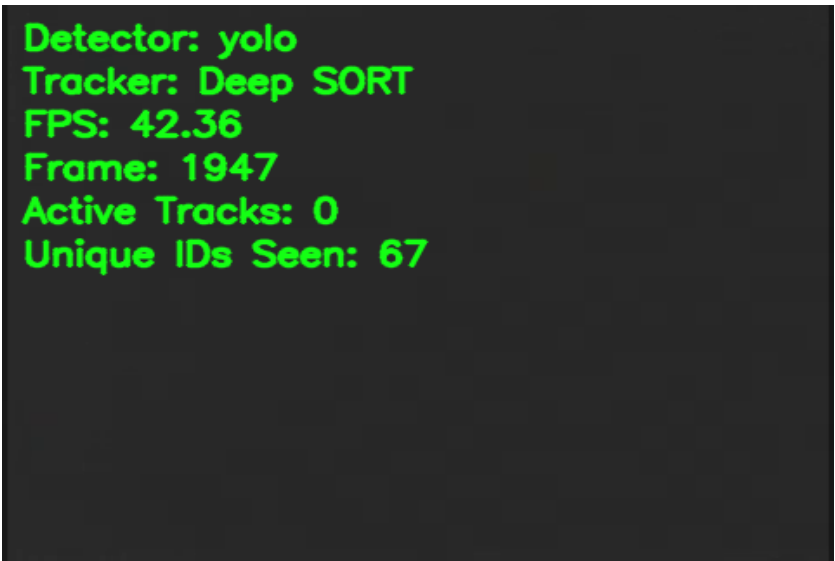


Figure 2. Live HUD overlay captured from the notebook output (final processed frame), showing the detector/tracker identification, smoothed FPS, frame counter, and the cumulative count of 67 unique object identities tracked across the full video.

```
Processing frame 1947 | Smoothed FPS: 42.36

Done. Output saved to: tracked_output.mp4
```

Figure 3. Final console output from the executed notebook cell.

9. Feature Summary

Feature	Description
Dual detector support	Swap between YOLO11 and Faster R-CNN via one config flag.
Persistent identity tracking	Deep SORT assigns and maintains stable IDs across frames.
Class filtering	Optional allow-list restricts tracking to specific object classes.
Per-ID color coding	Deterministic, visually distinct colors per tracked identity.
Motion trail visualization	Tapered trajectory lines show recent path and direction of travel.
Live analytics HUD	On-frame FPS, frame counter, active/unique track counts, per-class tallies.
Colab-native I/O	Built-in upload widget and inline video playback; also runs outside Colab.
GPU acceleration	Automatic CUDA detection with half-precision inference on GPU.
MP4 export	Full annotated run is saved as a standalone, shareable video file.

10. Limitations & Recommended Improvements

- **ID switches under heavy occlusion.** Long full occlusions beyond MAX_AGE still cause re-identification as a new ID; a stronger Re-ID embedder or longer NN_BUDGET could help.
- **Single-stream, offline processing.** The current loop processes one uploaded file end-to-end; a streaming/webcam-friendly buffered version would suit live deployments.
- **No persistence of analytics.** Per-class counts and trails reset each run; logging them to CSV/JSON would enable downstream analytics dashboards.
- **Faster R-CNN path is unbenchmarked here.** The notebook run captured used the YOLO backend only; a head-to-head accuracy/speed comparison against Faster R-CNN would strengthen the evaluation.
- **Fixed drawing scale.** Label and box scale are constants rather than resolution-adaptive, which can look small/large on very different input resolutions.

11. Conclusion

This task delivers a compact yet production-style object detection and tracking pipeline that cleanly separates detection, tracking, and visualization concerns while remaining fully configurable and Colab-ready. The successful run — 1,947 frames processed at an average smoothed throughput of 42.36 FPS while maintaining 67 distinct tracked identities — demonstrates that the system meets real-time performance targets on commodity Colab GPU hardware and produces a clear, analytics-rich annotated video suitable for demonstration or further downstream analysis.